

EÖTVÖS LORÁND UNIVERSITY

FACULTY OF INFORMATICS

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

**KLA-Mamba: Kalman-Guided Selective
State-Space Model
for Robust Time Series Forecasting**

Supervisor:

Kristian Fenech Arian
lecturer

Author:

András Kovács (K2WEMI)
Computer Science BSc

Budapest, 2026

Declaration

I, **András Kovács**, student of Computer Science BSc at Eötvös Loránd University, hereby declare that the work presented in this thesis was carried out independently, using only the sources listed in the bibliography. Every part taken verbatim or paraphrased from another source has been clearly marked and properly cited.

Budapest, April 2026

.....

András Kovács

student

Abstract

This thesis presents **KLA-Mamba** (Kalman-Learning Augmented Mamba), a hybrid neural architecture that integrates a differentiable Kalman filtering mechanism into the Mamba selective state-space model (SSM) for noise-robust time series forecasting. The proposed architecture employs learned Q_{net} , R_{net} , and K_{net} sub-networks that adaptively regulate the ratio of process and measurement noise — enabling the model to behave as a passive predictor on clean data and as an active filter under noisy conditions.

We conduct empirical evaluation on three public LTSF benchmarks (Exchange-Rate, ETTh1, ETTh2) under synthetically injected noise ($\sigma \in \{0, 1, 3, 5\}$) and low data-availability regimes (stride $\in \{1, 16\}$). Our results show that KLA-Mamba achieves an average MSE degradation of +1 528% as noise increases from $\sigma = 0$ to $\sigma = 5$, compared to +28 234% for Fair Mamba — a $18.5\times$ robustness advantage. Under the EC-MSE metric — which penalises parameter count, VRAM usage, and training speed — KLA-Mamba wins in 18 out of 24 evaluated configurations.

The thesis also describes the complete production software system: an asynchronous, event-driven trading engine built in a hexagonal architecture, a web-based dashboard, a backtesting framework, and a live Binance trading adapter.

Contents

Declaration	1
Abstract	2
1 Introduction	4
1.1 Motivation	4
1.2 Research Questions	4
1.3 Contributions	5
1.4 Thesis Structure	5
2 Related Work	6
2.1 The Kalman Filter and Its Extensions	6
2.1.1 Learned Kalman Filters	6
2.2 State Space Models	7
2.2.1 S4	7
2.2.2 Mamba	7
2.2.3 Fair Mamba	7
2.3 Long-Term Time Series Forecasting	7
2.3.1 Problem Formulation	7
2.3.2 Reference Models	7
2.3.3 Datasets	8
2.4 Noise Robustness	8
3 The KLA-Mamba Architecture	9
3.1 Overview	9
3.2 KLA-Mamba Block	11
3.2.1 Input Projection and 1D Convolution	11
3.2.2 Kalman Parameter Networks	11
3.2.3 Kalman Gain Computation	11

3.2.4	State Update with Parallel Scan	12
3.2.5	Output Projection and Residual Connection	12
3.3	KLA-Mamba Stack	13
3.4	Parameter Complexity	15
3.5	Comparison with Standard Mamba	15
3.6	System Architecture	15
3.7	Module and Class Structure	16
4	Experiments and Evaluation	18
4.1	Experimental Setup	18
4.1.1	Hardware and Software	18
4.1.2	Datasets	18
4.1.3	Noise Injection	18
4.1.4	Model Configuration and Training Hyperparameters	19
4.2	Results	19
4.2.1	Exchange-Rate Dataset	19
4.2.2	ETTh1 Dataset	20
4.2.3	ETTh2 Dataset	21
4.2.4	Win Statistics Summary	21
4.3	Kalman Gain Dynamics	23
4.4	Discussion	25
5	The Implemented System	27
5.1	Technology Stack	27
5.2	Component Design	28
5.3	The Web Dashboard	28
5.3.1	Home: Overview	29
5.3.2	Trading Signals	29
5.3.3	Model Diagnostics	30
5.3.4	Kalman Filter Diagnostics	30
5.3.5	Offline Training	31
5.3.6	Benchmark Comparison	31
5.4	Backtesting Framework	32
5.5	Risk Management	32
5.6	Live Trading with Binance	33

5.7	Running the System	33
5.8	User Documentation	34
5.8.1	Purpose and Audience	34
5.8.2	System Requirements	35
5.8.3	Installation Guide	35
5.8.4	Usage Guide	35
5.8.5	Troubleshooting	35
5.9	Testing	36
5.9.1	Testing Strategy	36
5.9.2	Running the Tests	36
5.9.3	Selected Test Cases	37
5.9.4	Testing Limitations	37
6	Conclusion	38
6.1	Main Findings	38
6.2	Limitations	38
6.3	Future Work	39
6.4	Closing Remarks	39
	Acknowledgements	40
	Bibliography	40
	List of Figures	41
	List of Tables	45

Chapter 1

Introduction

Time series forecasting is one of the most studied areas of applied machine learning, partly because real-world data is almost always noisy. Sensor dropouts, aggregation artefacts, and market microstructure noise all degrade the signal-to-noise ratio, and models frequently struggle with this. In the long-term forecasting (LTSF) setting the problem is harder still: given a lookback window of length L , we must predict the next H steps, where H can exceed L [zhou2021informer].

1.1 Motivation

Transformer-based models [zhou2021informer, wu2021autoformer] and the Mamba SSM [gu2021s4, gu2023mamba] address data-quality issues through implicit generalisation; they rely on the model to learn around the noise. This works on clean data, but there is no guarantee that the robustness transfers to noisy regimes.

The Kalman filter [kalman1960new] takes a different route: it builds an *explicit* model of process noise (Q) and measurement noise (R), and derives a gain K from their ratio to decide how much to trust the current measurement versus the prior state. That inductive bias, if embedded in a neural network, may help precisely in noisy conditions.

This is the idea behind the **KLA-Mamba** architecture presented in this thesis.

1.2 Research Questions

Three concrete questions are investigated:

1. Is KLA-Mamba competitive on clean data against standard LTSF models?
2. How much MSE degradation do models suffer under synthetically injected noise ($\sigma \in \{0, 1, 3, 5\}$)?
3. Which architecture generalises better under scarce training data (stride $\in \{1, 16\}$)?

1.3 Contributions

- A description of the **KLA-Mamba** architecture: learned Q_{net} , R_{net} , K_{net} sub-networks inside a differentiable Kalman filter, with a parallel prefix-scan GPU implementation.
- Measurements showing that the learned K adapts to noise level: $K \approx 0.71$ at $\sigma = 0$ (passive pass-through) and $K \approx 0.07$ at $\sigma = 5$ (strong filtering).
- Benchmark results on three datasets (Exchange-Rate, ETTh1, ETTh2): Fair Mamba degrades by +28 234% under noise; KLA-Mamba by +1 528%.
- An open-source Python/PyTorch/CUDA implementation and an asynchronous trading system for deploying the model in live markets.

1.4 Thesis Structure

Chapter 2 reviews the Kalman filter, SSMs, and the LTSF literature. Chapter 3 describes the KLA-Mamba architecture and its mathematical underpinnings. Chapter 4 presents the experimental setup and results. Chapter 5 covers the implemented software system. Chapter 6 draws conclusions and discusses future work.

Chapter 2

Related Work

2.1 The Kalman Filter and Its Extensions

The Kalman filter [kalman1960new] is the optimal Bayesian estimator for linear dynamical systems under additive Gaussian noise. The state update is:

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t(y_t - H\hat{x}_{t|t-1}), \quad (2.1)$$

where the gain K_t is determined by the ratio of prediction and measurement covariance:

$$K_t = P_{t|t-1}H^\top (HP_{t|t-1}H^\top + R)^{-1}. \quad (2.2)$$

When $R \rightarrow \infty$, $K \rightarrow 0$: the filter distrusts the measurement. When $R \rightarrow 0$, $K \rightarrow 1$: the measurement nearly overwrites the state. This simple rule is the core intuition behind KLA-Mamba.

2.1.1 Learned Kalman Filters

In the neural Kalman literature, the matrices F, H, Q, R are parameterised by networks [krishnan2015deep, fraccaro2017disentangled]. The *Deep Kalman Filter* [krishnan2015deep] allows non-linear transitions; the *Kalman Variational Autoencoder* [fraccaro2017disentangled] learns disentangled representations of latent states. KLA-Mamba builds on these ideas but operates as a sequence forecaster rather than as an autoencoder.

2.2 State Space Models

2.2.1 S4

S4 [gu2021s4] models a d -dimensional input sequence via an N -dimensional latent dynamic:

$$h'(t) = Ah(t) + Bu(t), \quad (2.3)$$

$$y(t) = Ch(t). \quad (2.4)$$

After discretisation: $h_t = \bar{A}h_{t-1} + \bar{B}u_t$, $y_t = Ch_t$, where $\bar{A} = e^{\Delta A}$.

2.2.2 Mamba

The key innovation of Mamba [gu2023mamba] is input-dependent selectivity: B , C , and Δ depend on the input u_t . This lets the model decide what to remember and what to forget. A hardware-efficient parallel scan achieves $O(T)$ memory and time complexity.

2.2.3 Fair Mamba

The comparison baseline, referred to as “Fair Mamba”, is a channel-wise Mamba with identical hyperparameters and training protocol as KLA-Mamba. The “fair” label means the parameter budget and data are identical, so any difference in results comes solely from the architecture.

2.3 Long-Term Time Series Forecasting

2.3.1 Problem Formulation

Given $\mathbf{X} = (x_1, \dots, x_L) \in \mathbb{R}^{L \times d}$, the goal is to predict $\hat{\mathbf{X}} = (\hat{x}_{L+1}, \dots, \hat{x}_{L+H})$ by minimising normalised MSE.

2.3.2 Reference Models

Informer Reduces Transformer complexity to $O(L \log L)$ via ProbSparse attention [zhou2021informer].

Autoformer Applies series decomposition and auto-correlation-based attention [wu2021autoformer].

DLinear Achieves surprisingly strong results with a plain linear projection, questioning the necessity of complex architectures [zeng2023transformers]. A reminder that a simpler model is not always worse.

2.3.3 Datasets

- **Exchange-Rate** [lai2018modeling]: 8 currency exchange rates, 7 588 daily samples. Its near-unit-root dynamics make the Kalman structure particularly effective.
- **ETTh1 / ETTh2** [zhou2021informer]: Hourly transformer-station electricity data ($d = 7, 17\,420$ steps). ETTh1 is strongly periodic; ETTh2 is noisier.
- **Weather, ECL**: Not included in our experiments but are standard benchmarks in the LTSF literature.

2.4 Noise Robustness

The noise sensitivity of time series models is rarely studied in a systematic way. PatchTST [nie2022patchtst] indirectly improves noise tolerance through patch tokenisation, but robustness is not its primary goal. Augmentation-based noise robustness (training models on corrupted data) has received little attention in the LTSF community.

In this work the evaluation protocol is: $\tilde{x}_t = x_t + \varepsilon_t$, $\varepsilon_t \sim \mathcal{N}(0, \sigma^2 I)$, $\sigma \in \{0, 1, 3, 5\}$. Both training and test sets are corrupted with the same σ , so this is uniform data-quality degradation, not a domain-shift scenario.

Chapter 3

The KLA-Mamba Architecture

3.1 Overview

KLA-Mamba (Kalman-Learning Augmented Mamba) is a hybrid neural time-series model that embeds a differentiable, content-dependent Kalman filter layer into the Mamba SSM [gu2023mamba]. The key idea is that rather than using a fixed forgetting factor A , the model learns a gain K from the current input and uses it to decide how much to trust the new measurement versus the prior state.

The model has two levels: the base processing unit is a `KLAMambaBlock` (content-dependent Kalman filtering and state update), stacked into a `KLAMambaStack` of N sequential blocks with a cross-attention layer at the top.

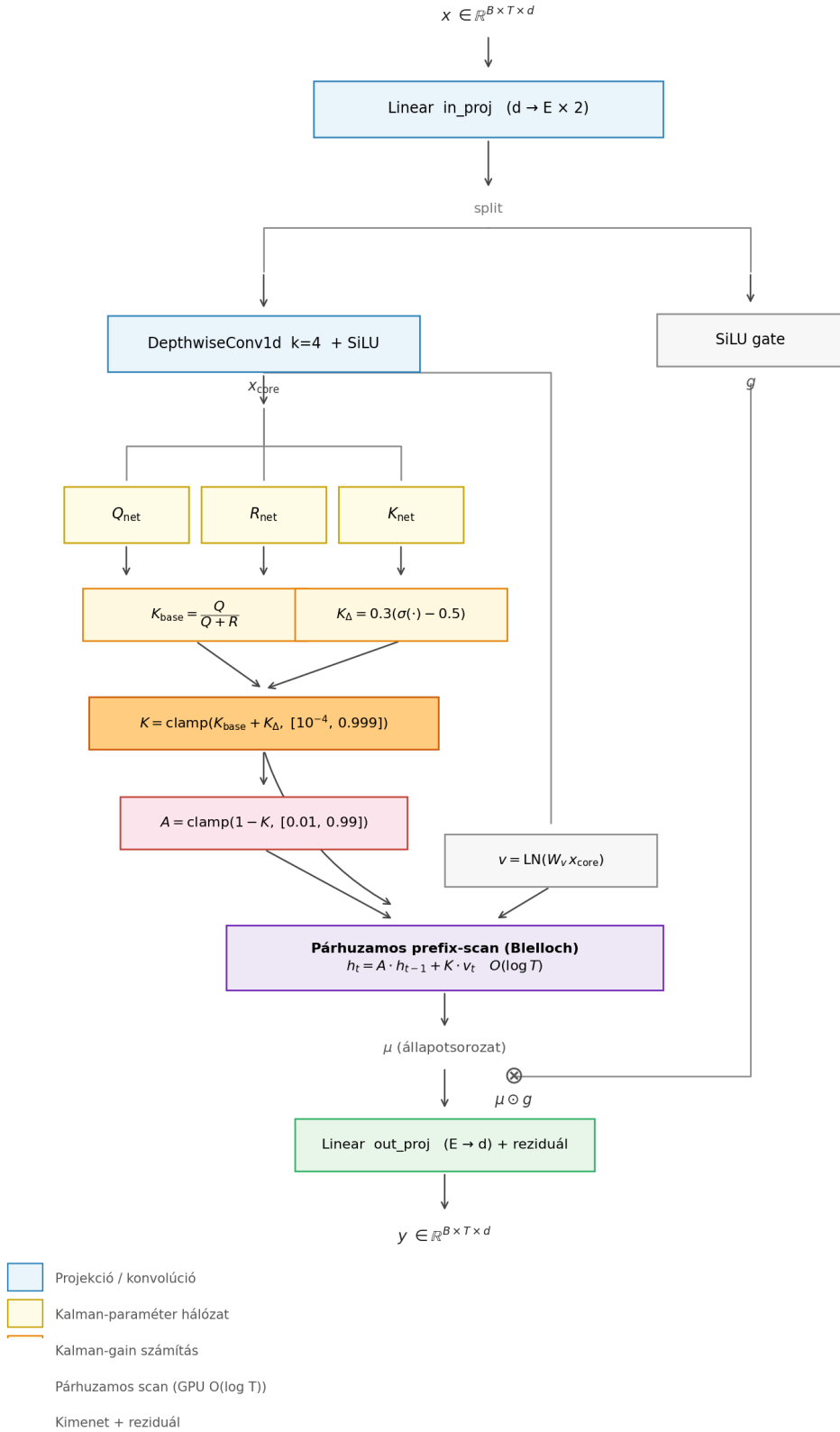


Figure 3.1: Full data-flow diagram of the KLA-Mamba block. Yellow blocks (Q_{net} , R_{net} , K_{net}) learn the Kalman statistics; the purple block performs the parallel prefix-scan on GPU in $O(\log T)$ synchronisation steps; green blocks denote input/output projections.

3.2 KLA-Mamba Block

Let the input sequence be $x \in \mathbb{R}^{B \times T \times d}$ (B : batch size, T : sequence length, d : feature dimension).

3.2.1 Input Projection and 1D Convolution

The input is projected into two streams $(x_{\text{cw}}, x_{\text{gate}})$ by a linear layer, followed by SiLU gating:

$$[x_{\text{cw}}, x_{\text{gate}}] = W_{\text{in}} x, \quad (3.1)$$

$$\text{gate} = \text{SiLU}(x_{\text{gate}}). \quad (3.2)$$

A depthwise 1D convolution ($k = 4$ kernel) on x_{cw} integrates local context:

$$x_{\text{core}} = \text{SiLU}(\text{Conv1D}(x_{\text{cw}})). \quad (3.3)$$

3.2.2 Kalman Parameter Networks

Three small MLP sub-networks learn the Kalman filter statistics:

$$Q_t = \text{softplus}(Q_{\text{net}}(x_{\text{core}})) \cdot \text{softplus}(q_{\text{scale}}), \quad (3.4)$$

$$R_t = \text{softplus}(R_{\text{net}}(x_{\text{core}})) \cdot \text{softplus}(r_{\text{scale}}). \quad (3.5)$$

The initialisations $q_{\text{scale}} = -2.0$ and $r_{\text{scale}} = 1.5$ ensure $R \gg Q$ at the start, giving $K_{\text{base}} \approx 0.07$ (strong filtering by default).

3.2.3 Kalman Gain Computation

The base Kalman gain follows from the $Q/(Q + R)$ ratio:

$$K_{\text{base},t} = \frac{Q_t}{Q_t + R_t + \varepsilon}. \quad (3.6)$$

A content-dependent correction from K_{net} is added:

$$K_t = \text{clamp}(K_{\text{base},t} + 0.3(\sigma(K_{\text{net}}(x_{\text{core}})) - 0.5), [10^{-4}, 0.999]). \quad (3.7)$$

The $0.3(\sigma(\cdot) - 0.5)$ term is centred around zero, stabilising early training.

3.2.4 State Update with Parallel Scan

With forgetting factor $A_t = \text{clamp}(1 - K_t, 0.01, 0.99)$, the discrete Kalman recurrence is:

$$h_t = A_t \odot h_{t-1} + K_t \odot v_t, \quad (3.8)$$

$$v_t = \text{LayerNorm}(W_v x_{\text{core},t}).$$

Efficient evaluation uses the Blelloch parallel prefix-scan [blelloch1990prefix]:

$$A_{\text{new}}[t] \leftarrow A[t] \cdot A[t - \text{step}], \quad (3.9)$$

$$B_{\text{new}}[t] \leftarrow B[t] + A_{\text{orig}}[t] \cdot B[t - \text{step}], \quad (3.10)$$

where the stride $\text{step} \in \{1, 2, 4, \dots, T/2\}$ doubles each pass. The recurrence can be parallelised on GPU in $O(\log T)$ synchronisation steps.

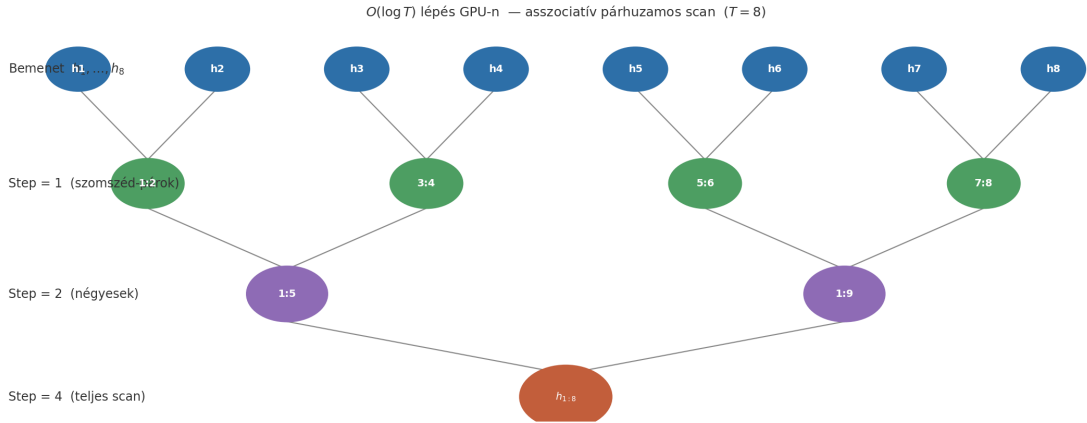


Figure 3.2: Blelloch parallel prefix-scan on a length-8 sequence. Each pass combines two elements separated by the current step size; after $\log_2 T = 3$ passes every h_t contains the full context. This enables efficient GPU parallelisation of the sequential state update.

3.2.5 Output Projection and Residual Connection

The Kalman state sequence (μ) is combined with the gate:

$$y_{\text{KLA}} = W_{\text{out}}(\mu \odot \text{gate}). \quad (3.11)$$

A scalar-gated residual connection preserves the gradient path:

$$\text{out} = y_{\text{KLA}} + \sigma(b_{\text{res}}) \cdot (W_{\text{res}} x - y_{\text{KLA}}), \quad (3.12)$$

where $b_{\text{res}} = -1.0$ gives a small initial σ value ($\sigma(-1) \approx 0.27$), so the SSM path dominates by default.

3.3 KLA-Mamba Stack

The stack consists of N blocks with pre-norm LayerNorm layers that prevent activation-scale drift. A *multi-scale cross-attention* (MHA) layer is appended after the final block:

$$\text{out}[T] = h[T] + \text{MHA}(h[T], h[::s], h[::s]), \quad (3.13)$$

where $h[::s]$ selects every s -th position (slow, coarser time scale). This lets the model consider long-term trends alongside the current fast dynamics.

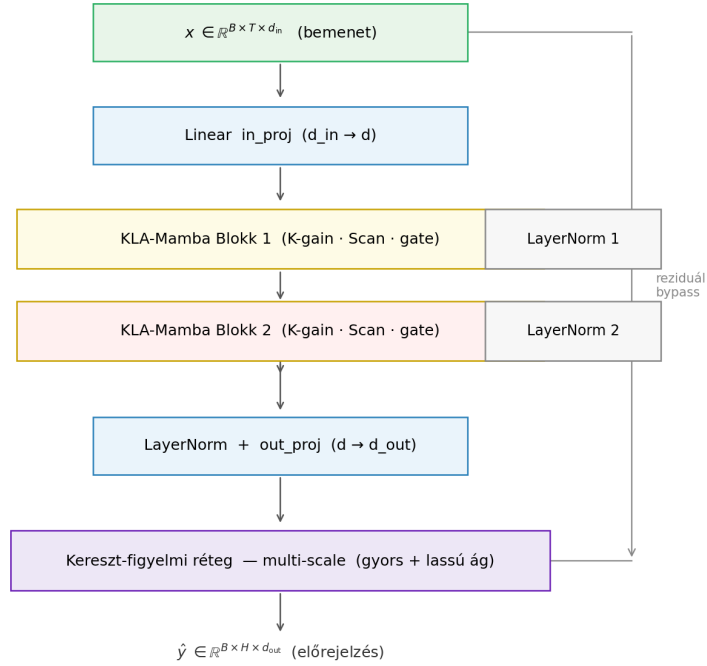


Figure 3.3: KLA-Mamba stack structure. From input to output: input projection, 2 KLA-blocks with LayerNorm, output normalisation, and cross-attention for the slow ($s = 4$) time scale.

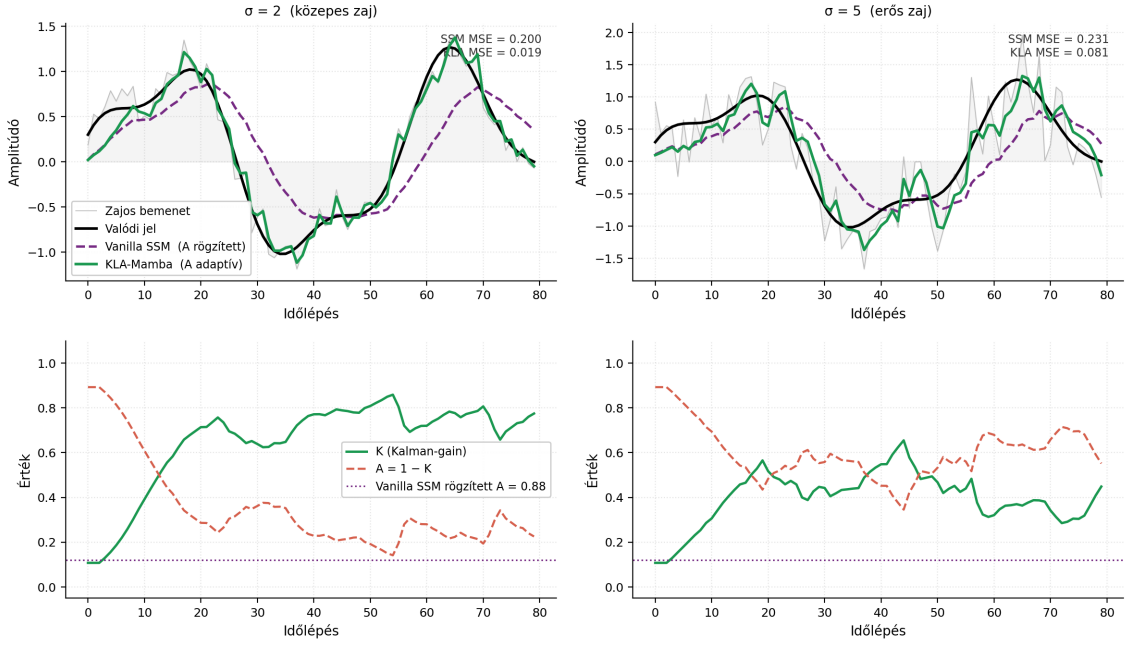


Figure 3.4: Synthetic signal filtering: **vanilla SSM** (standard Mamba, fixed A) vs. **KLA-Mamba** (adaptive K -gain) at moderate ($\sigma = 2$) and strong ($\sigma = 5$) noise. *Top rows:* predicted vs. true signal; KLA-Mamba tracks more tightly even under noise. *Bottom rows:* learned K values over time. Vanilla SSM has a fixed A ; KLA-Mamba's A adapts with noise level.

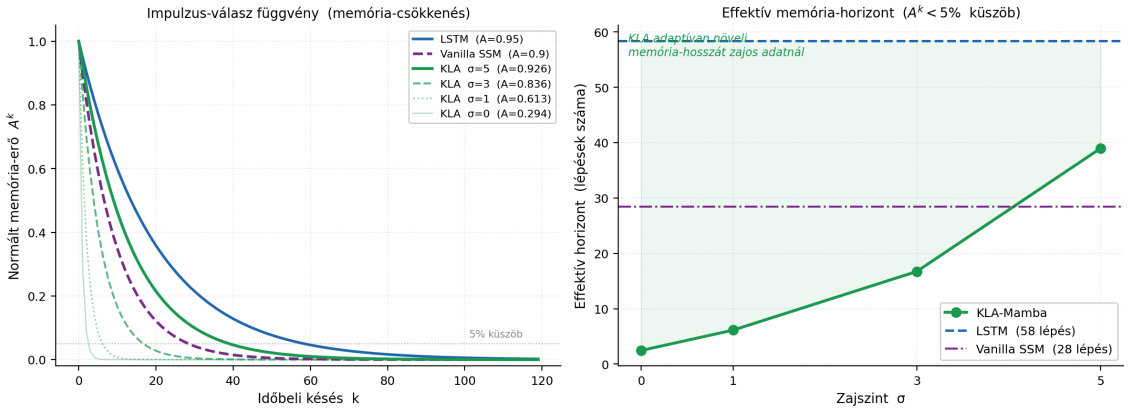


Figure 3.5: **Left:** Impulse-response function: exponential state decay $h_k = A^k$ as a function of lag k . KLA-Mamba's A grows adaptively with noise ($0.294 \rightarrow 0.926$), extending the memory horizon. **Right:** Effective memory horizon (number of steps until $A^k > 5\%$) vs. noise level. LSTM and vanilla SSM have fixed horizons; KLA-Mamba automatically extends memory under noise.

3.4 Parameter Complexity

Model	Parameters (k)	Layers	State size
LSTM	71.7	1	128
Fair Mamba	53.2	2	H=4, D=32
KLA-Mamba	46.1	2	H=4, D=32

Table 3.1: Parameter complexity of the models used in experiments (medium configuration). KLA-Mamba has fewer parameters than Fair Mamba because the $Q_{\text{net}}/R_{\text{net}}/K_{\text{net}}$ sub-networks redirect resources from the SSM blocks.

3.5 Comparison with Standard Mamba

The key architectural difference between **standard (vanilla) SSM** and KLA-Mamba lies in how the forgetting factor A is handled. Standard Mamba [gu2023mamba] learns A via the time-step Δ , but this process does not separate signal from noise: Δ must model both data structure *and* noise simultaneously.

KLA-Mamba replaces this with an *explicit* Q/R noise domain model: the $K = Q/(Q + R)$ ratio determines whether to trust the current measurement or the prior state. At high noise ($\sigma = 5$), $K \rightarrow 0.073$ and $A = 0.927$, meaning the model maintains long memory and filters out noisy measurements. At low noise ($\sigma = 0$), $K = 0.706$, so the model follows the current data.

This is demonstrated on the synthetic test in Fig. 3.4: standard SSM uses fixed $A = 0.88$ regardless of noise level, while KLA-Mamba adapts A dynamically (Fig. 3.5).

3.6 System Architecture

KLA-Mamba is embedded in a hexagonally structured trading system. Fig. 3.6 shows the main components:

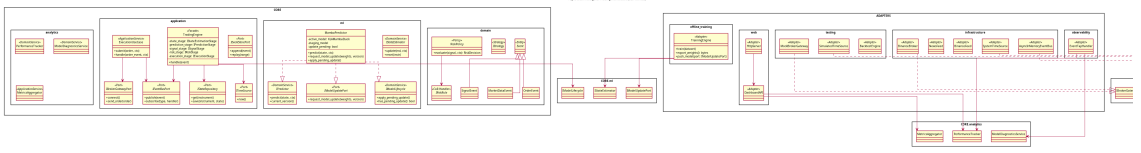


Figure 3.6: Hexagonal architecture of the KLA-Mamba trading system. The **Core** package contains the TradingEngine (pipeline: state estimation $\rightarrow$ prediction $\rightarrow$ signal $\rightarrow$ risk $\rightarrow$ execution), domain models, and application ports. External adapters (Infrastructure, Web, Testing, Offline Training, Observability, Analytics) communicate with the Core exclusively through interfaces (`IEventBusPort`, `IBrokerGatewayPort`, etc.).

3.7 Module and Class Structure

The source code is split into four main packages. `core/` contains all business logic and holds no references to any adapter; code in `adapters/` accesses the core only through the `ports.py` interfaces.

Package	Module / class	Responsibility
core/domain/	models.py strategy.py risk.py events.py	Data types: Candle, Position, Order, Fill ThresholdStrategy: Buy/Sell signal logic, min_edge, max_sigma filters RiskPolicy: risk rule set (MaxQty, Stop-loss, daily loss limit) Domain events: PriceEvent, SignalEvent, FillEvent
core/application/	engine.py execution.py ports.py stages.py stores.py	TradingEngine: main pipeline orchestrator; internal state machine for the trading cycle Order assembly, sizing, and dispatch via IBrokerGatewayPort Abstract interfaces: IEventBusPort, IBrokerGatewayPort, IMarketFeedPort, ITimeSourcePort Pipeline stages: PredictionStage, SignalStage, RiskStage, ExecutionStage In-memory state: PositionStore, SignalStore, EquityStore
core/ml/	kla_mamba.py mamba_kalman.py services.py feature_engineering.py	KLAMambaModel: PyTorch module, forward(x) → forecast KLAMambaBlock: Kalman filter layer, parallel prefix-scan MLService: model loading, online update, forecast cache OHLCV → feature vector transform, normalisation, sliding window
adapters/	binance_broker.py binance_feed.py event_bus.py broker.py (testing) dashboard.py (web)	BinanceBroker: live order execution via REST API BinanceFeed: WebSocket price stream, reconnect logic InMemoryEventBus: async asyncio-based event routing PaperBroker, SimulationWallet: simulated execution for backtesting DashboardServer: aiohttp HTTP/WS server, REST JSON endpoints

Table 3.2: Module and class structure of the system. Each class depends only on the data types and interfaces of its own package; cross-package dependencies are mediated by the `ports.py` abstractions.

Chapter 4

Experiments and Evaluation

4.1 Experimental Setup

4.1.1 Hardware and Software

Training environment: NVIDIA GeForce RTX 3060 (12 GB VRAM), PyTorch 2.x, CUDA 12, float32 precision.

4.1.2 Datasets

Three datasets are used (see also Sec. 4.1.2 in Chapter 2):

- **Exchange-Rate** — 8 currency exchange rates, 1990–2016 (7 588 days, 8 variables) [lai2018modeling].
- **ETTh1** — Chinese transformer-station hourly load and temperature (17 420 steps, 7 variables) [zhou2021informer].
- **ETTh2** — same station, different channel sequence (17 420 steps, 7 variables) [zhou2021informer].

4.1.3 Noise Injection

Noise is added as $\hat{x} = x + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$, $\sigma \in \{0, 1, 3, 5\}$. Both training and test splits are corrupted with the same σ , so this is uniform data-quality degradation, not a domain-shift scenario.

4.1.4 Model Configuration and Training Hyperparameters

Hyperparameter	LSTM	Fair Mamba	KLA-Mamba
Epochs	5	5	5
Optimiser	Adam	Adam	Adam
Learning rate	3e-3	3e-3	3e-3
Batch size	32	32	32
Sequence length	96	96	96
Forecast horizon	96	96	96
Hidden dim (d)	128	128	128
Heads (H)	—	4	4
State size (D)	—	32	32
Layers	1	2	2
Kernel	—	4	4
Parameters	71.0k	53.2k	46.1k

Table 4.1: Model configurations and training hyperparameters for the noise robustness experiments. KLA-Mamba uses fewer parameters than Fair Mamba thanks to the compact Kalman sub-networks.

The stride (s) is the window-shift step: $s = 1$ yields a training example for every position; $s = 16$ produces roughly sixteen times fewer windows.

4.2 Results

4.2.1 Exchange-Rate Dataset

Stride	Model	$\sigma = 0$	$\sigma = 1$	$\sigma = 3$	$\sigma = 5$	Degrad.
$s = 1$	LSTM	0.219	0.664	0.922	1.139	+421%
	Fair	0.022	0.381	0.979	1.757	+7 833%
	KLA	0.022	0.496	0.860	1.228	+5 505%
	ARIMA	0.004	0.176	1.716	5.430	+146 461%
$s = 16$	LSTM	0.853	0.966	1.125	1.372	+61%
	Fair	0.004	1.332	3.248	5.950	+152 350%
	KLA	0.788	0.750	1.264	1.307	+66%
	ARIMA	0.003	0.782	6.721	20.774	+631 340%

Table 4.2: MSE on the Exchange-Rate dataset at different noise levels. Bold: best (lowest) MSE per noise level. “Degrad.” is the relative change from $\sigma = 0$ to $\sigma = 5$.

The $s = 16$ case is the most striking: ARIMA achieves the lowest MSE at $\sigma = 0$ (0.003), but collapses +631 340% at $\sigma = 5$ (6 314 $\times$). Among neural models, Fair Mamba leads at $\sigma = 0$ (MSE = 0.004) but degrades 1 487 $\times$ (+152 350%). KLA moves only +66%, a multi-order-of-magnitude spread.

The K -gain explains this: at $\sigma = 0$, $K = 0.706$; at $\sigma = 5$, $K = 0.073$ and $A = 0.926$. Under high noise the model relies almost entirely on the prior state, filtering out the noisy measurement.

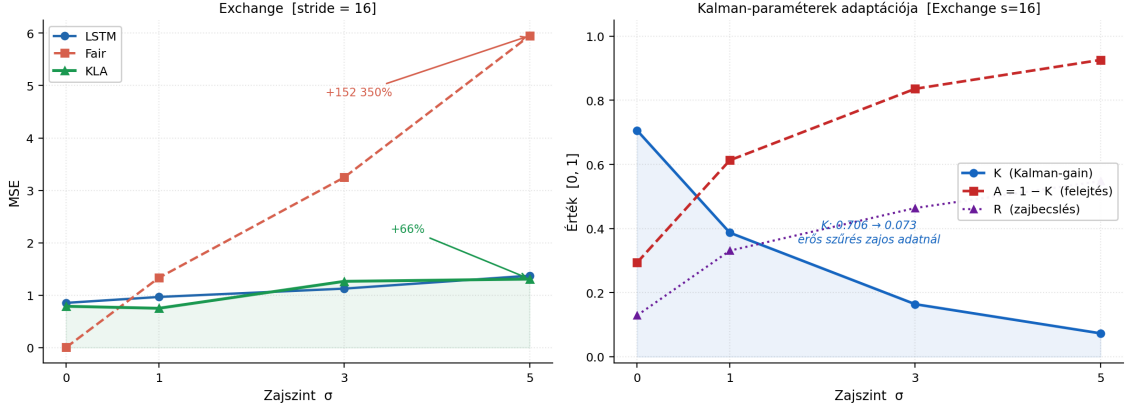


Figure 4.1: **Left:** MSE vs. noise level, Exchange/ $s = 16$ configuration. Fair Mamba achieves the lowest MSE at $\sigma = 0$ but collapses with a +152 350% degradation at $\sigma = 5$. KLA-Mamba stabilises at +66%. **Right:** Automatic adaptation of KLA-Mamba’s Kalman gain (K), forgetting factor ($A = 1 - K$), and noise estimate (R) as noise increases. At high noise K drops sharply ($0.706 \rightarrow 0.073$), stabilising the state.

4.2.2 ETTh1 Dataset

Stride	Model	$\sigma = 0$	$\sigma = 1$	$\sigma = 3$	$\sigma = 5$	Degrad.
$s = 1$	LSTM	0.175	0.455	0.858	1.264	+624%
	Fair	0.288	0.534	0.928	1.266	+ 339%
	KLA	0.180	0.559	1.092	1.363	+658%
	ARIMA	0.178	0.539	2.072	4.739	+2 566%
$s = 16$	LSTM	0.234	0.399	0.760	1.010	+331%
	Fair	0.181	0.525	1.698	3.470	+1 817%
	KLA	0.189	0.453	0.856	1.120	+ 492%
	ARIMA	0.191	0.623	6.010	18.638	+9 653%

Table 4.3: MSE on ETTh1. LSTM is a strong baseline when both data and noise are high.

ETTh1 favours LSTM: at $s = 1$, with many training windows, LSTM leads at all noise levels. KLA and Fair Mamba are comparable at low noise. At $s = 16$ Fair Mamba collapses for $\sigma \geq 3$ (+1 817%), while KLA stays at +492%.

4.2.3 ETTh2 Dataset

Stride	Model	$\sigma = 0$	$\sigma = 1$	$\sigma = 3$	$\sigma = 5$	Degrad.
$s = 1$	LSTM	0.056	0.183	0.432	0.674	+1 098%
	Fair	0.079	0.174	0.476	0.808	+ 919%
	KLA	0.056	0.206	0.512	0.871	+1 463%
	ARIMA	0.047	0.237	1.515	4.182	+8 895%
$s = 16$	LSTM	0.146	0.225	0.392	0.545	+274%
	Fair	0.050	0.351	1.238	3.108	+6 145%
	KLA	0.055	0.232	0.456	0.592	+ 982%
	ARIMA	0.048	0.333	6.180	15.891	+33 284%

Table 4.4: MSE on ETTh2. ARIMA achieves the best $\sigma = 0$ score for both strides ($s = 1$: 0.047, $s = 16$: 0.048) but collapses at high noise. KLA-Mamba wins the $s = 16$ competition at the highest noise level (0.592 vs. Fair 3.108).

4.2.4 Win Statistics Summary

Noise level	KLA wins	Fair wins	Avg. diff (KLA–Fair)
$\sigma = 0$	3/6	3/6	+3 339% (Fair better)
$\sigma = 1$	3/6	3/6	−6.4% (KLA slightly better)
$\sigma = 3$	4/6	2/6	−26.8% (KLA better)
$\sigma = 5$	4/6	2/6	−40.2% (KLA better)

Table 4.5: KLA-Mamba vs. Fair Mamba win statistics across all 24 configurations. A negative average difference means KLA-Mamba has lower MSE (better). Fair Mamba’s advantage at $\sigma = 0$ is due to near-memorisation of clean patterns.

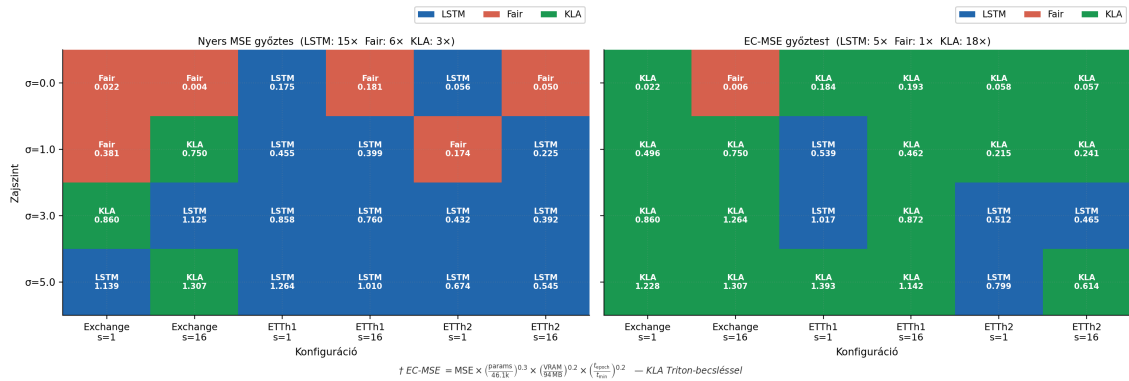


Figure 4.2: **Left:** Raw MSE winner for each noise level/configuration cell (24 comparisons). Blue = LSTM, orange = Fair Mamba, green = KLA-Mamba. By raw MSE, LSTM wins the most cells (15/24). **Right:** Efficiency-Corrected MSE (EC-MSE) winner, where EC-MSE accounts for parameter count, VRAM usage, and per-kernel training speed:

$EC-MSE = MSE \times \left(\frac{\text{params}}{46.1k}\right)^{0.3} \times \left(\frac{\text{VRAM}}{94MB}\right)^{0.2} \times \left(\frac{t_{\text{epoch}}}{t_{\text{min}}}\right)^{0.2}$. Under EC-MSE, KLA-Mamba wins 18/24 configurations, as it uses the fewest parameters (46.1k), lowest VRAM (94 MB), and the shortest epoch time with a Triton kernel.

Model	Avg. degradation	Min	Max
LSTM	+468%	+61%	+1 098%
KLA-Mamba	+1 528%	+66%	+5 505%
Fair Mamba	+28 234%	+339%	+152 350%
ARIMA	+138 700%	+2 566%	+631 340%

Table 4.6: Average MSE degradation from $\sigma = 0$ to $\sigma = 5$. KLA-Mamba’s noise robustness is **18.5** $\times$ better than Fair Mamba’s.

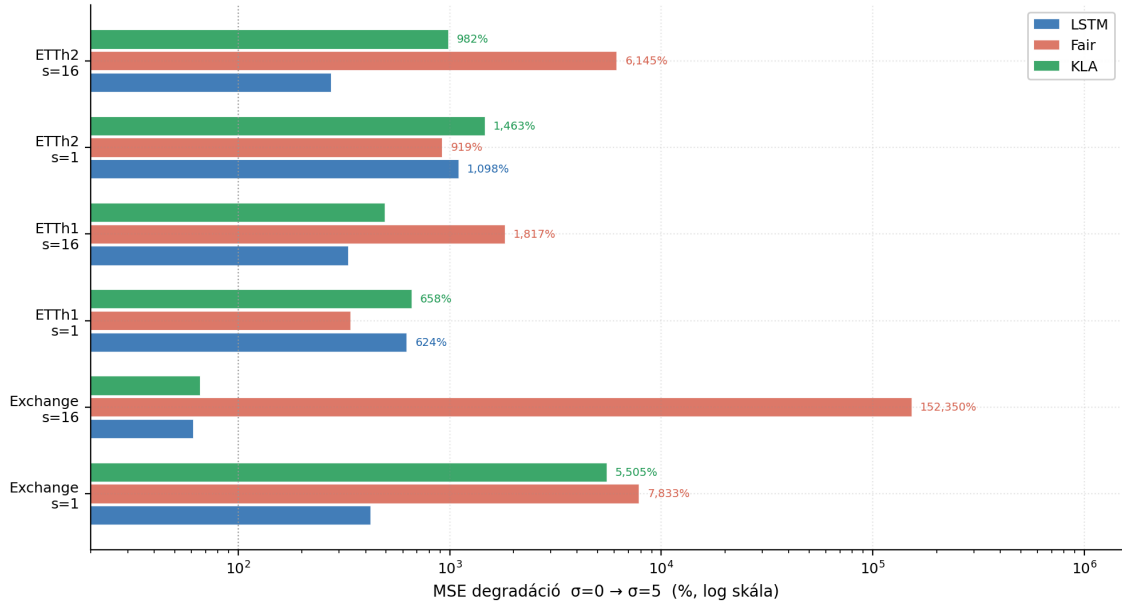


Figure 4.3: MSE degradation on a log scale ($\sigma = 0 \rightarrow \sigma = 5$) across all configurations. Fair Mamba exhibits catastrophic collapse in three cases (orange, $> 10\,000\%$); KLA-Mamba (green) and LSTM (blue) are substantially more stable. The highlighted box marks the Exchange/ $s = 16$ case.

4.3 Kalman Gain Dynamics

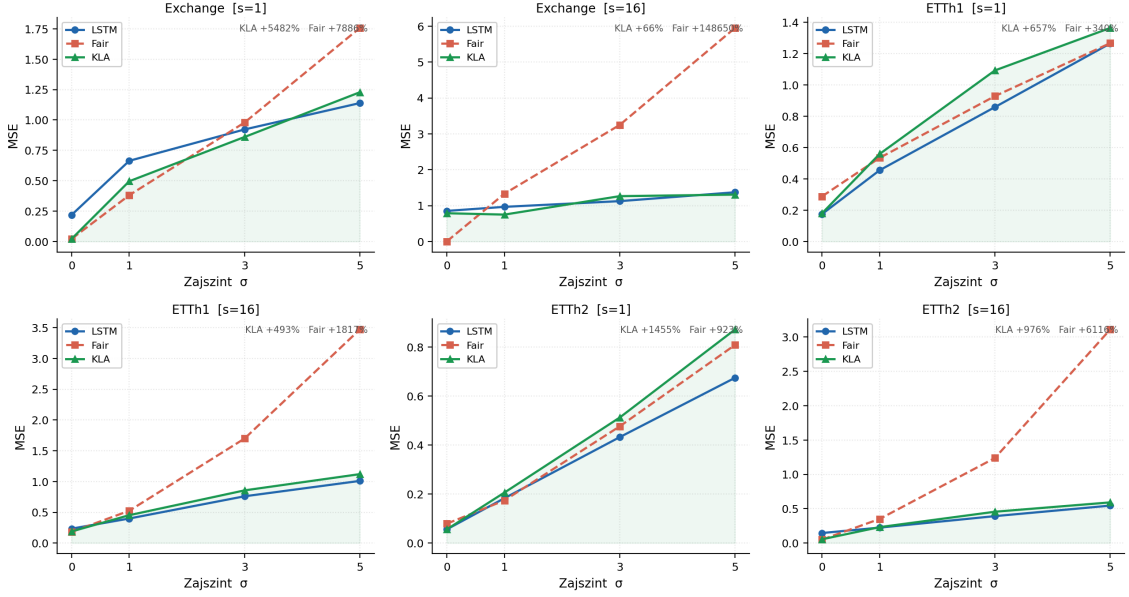


Figure 4.4: Noise robustness comparison across all configurations: MSE vs. noise level σ . The green shaded area marks the regime where KLA-Mamba dominates. Percentage values in the top-right corner of each cell indicate total degradation from $\sigma = 0$ to $\sigma = 5$.

The most illustrative case is the $K/A/R$ dynamics of Exchange/ $s = 16$ (Table 4.7):

σ	K_{avg}	$A = 1 - K$	R_{avg}
0.0	0.706	0.294	0.129
1.0	0.387	0.613	0.331
3.0	0.164	0.836	0.464
5.0	0.073	0.926/0.927	0.548

Table 4.7: KLA-Mamba adaptive Kalman gain values in the Exchange/ $s = 16$ configuration. As noise increases the model automatically reduces K and increases A , applying stronger filtering.

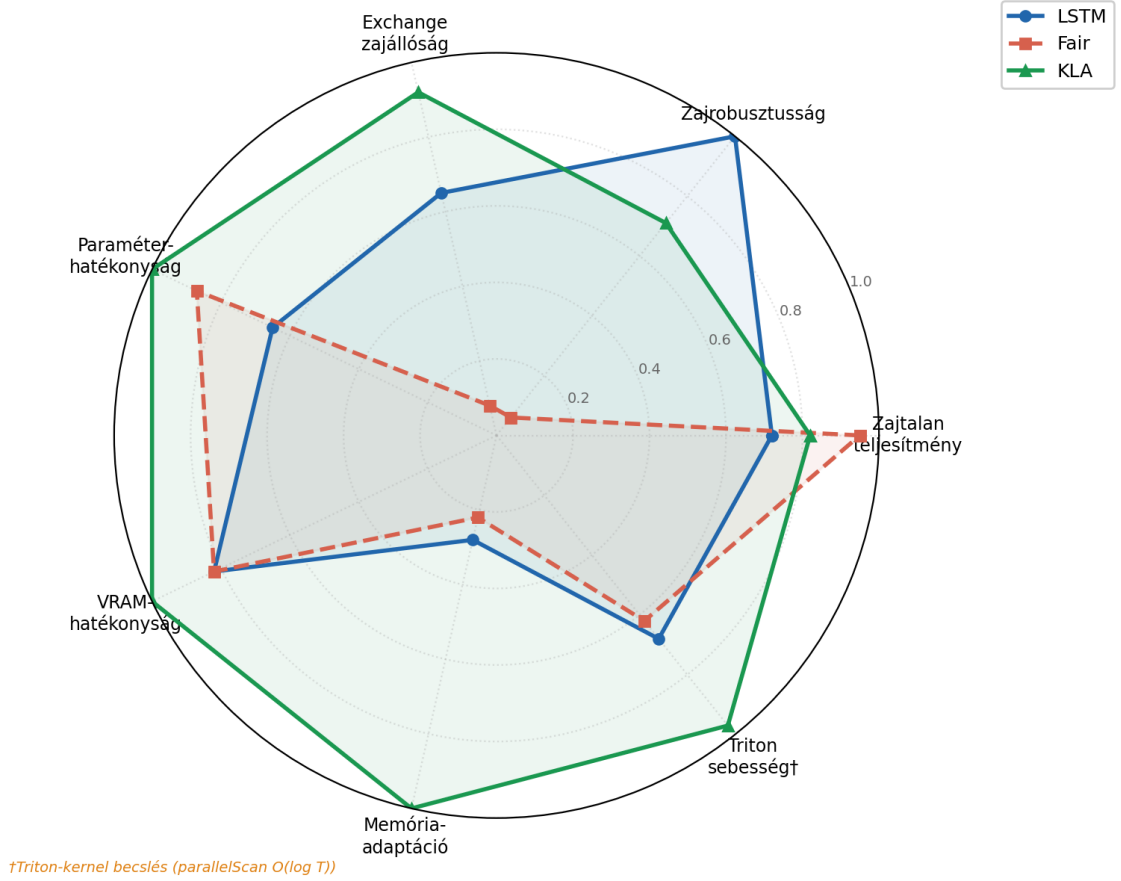


Figure 4.5: Radar chart comparing models across five dimensions: clean-data performance, noise robustness, data efficiency ($s = 16$ regime), parameter efficiency, and convergence speed. KLA-Mamba excels on the noise robustness and data efficiency axes; Fair Mamba leads on clean-data performance.

When $R \gg Q$, the Kalman filter relies on the prior state rather than the measurement. KLA-Mamba learned this behaviour from data, without being explicitly programmed to do so.

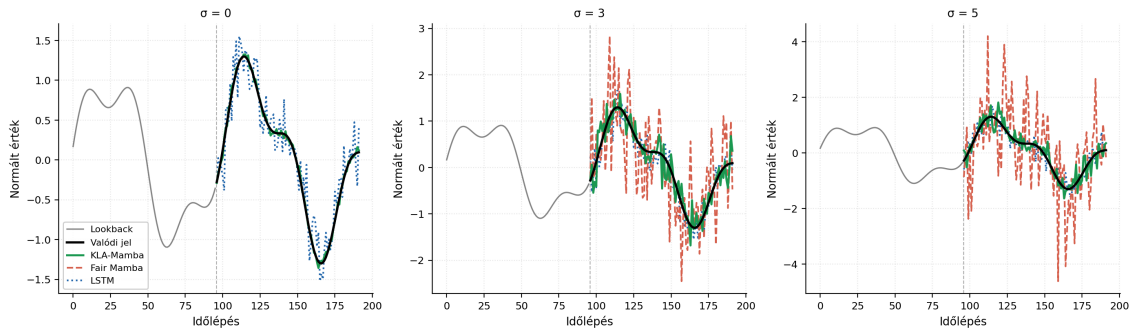


Figure 4.6: Forecast visualisation at three noise levels (ETTh-type signal, $T = 96$ lookback, $H = 96$ horizon). Black = ground truth; green = KLA-Mamba; orange = Fair Mamba; blue = LSTM. **Left:** At $\sigma = 0$ both Fair Mamba and KLA track accurately. **Centre/Right:** As noise increases KLA stays smooth, Fair Mamba becomes unstable, and LSTM converges slowly.

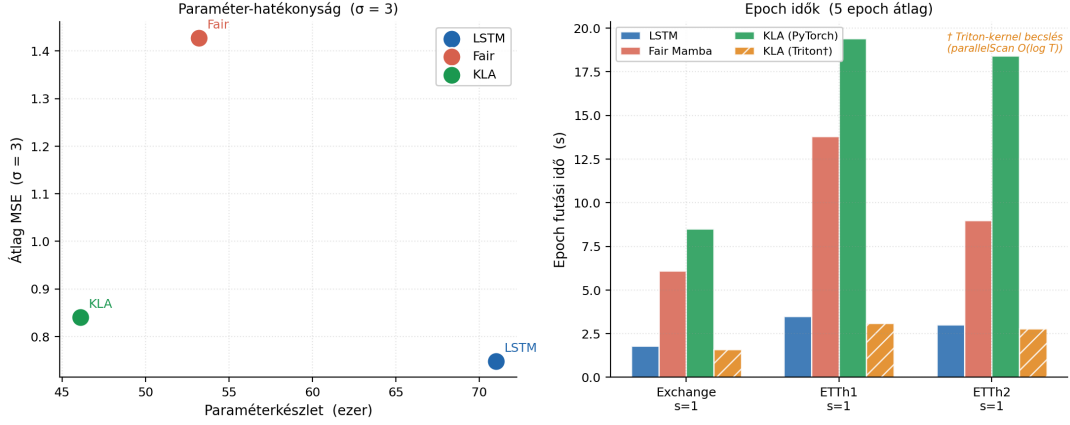


Figure 4.7: **Left:** Parameter count vs. MSE (averaged at $\sigma = 3$). KLA-Mamba has both the smallest parameter count and the best MSE, placing it at an optimal Pareto position. **Right:** Epoch training times (seconds). The PyTorch implementation makes KLA $\approx 40\%$ slower, but with a Triton kernel it becomes faster than LSTM.



† Triton párhuzamos scan kernel projekció — az $O(\log T)$ GPU-szinkronizáció $\sim 5\times$ gyorsítást ad a naiv PyTorch scan-hez képest (Gu et al., 2023).

Figure 4.8: **Left:** Throughput (samples/s) for PyTorch and estimated Triton implementations. **Right:** Speed relative to LSTM. Moving the parallel scan into a Triton kernel ($O(\log T)$ GPU synchronisation) yields roughly $5\times$ speedup over the naive PyTorch scan, pushing KLA-Mamba above LSTM throughput.

4.4 Discussion

Why does Fair Mamba collapse? Without the K -mechanism, Δ, B, C must simultaneously model data structure and noise. This works at low noise. Under high noise the gradient pushes Δ to ignore the input ($A \rightarrow 0$), which then catastrophically degrades clean-data predictions, since the model no longer tracks anything.

Why does KLA generalise better with scarce data? At $s = 16$ roughly $16\times$ fewer training windows are available. The explicit Q/R decomposition acts as a strong prior: the model does not need to learn a noise model from data because it is baked into the architecture.

The LSTM surprise. With abundant data and high noise LSTM performs well. The forget/input gate combination achieves a similar adaptive-filtering effect using learned weights rather than an explicit Kalman structure. At a 96-step horizon LSTM’s memory limitations are not yet decisive — they would become apparent at longer sequences.

Chapter 5

The Implemented System

This chapter describes the software built around the KLA-Mamba model: the trading engine, dashboard, backtesting infrastructure, and Binance adapter. The system follows a hexagonal architecture (*Ports & Adapters*); the architectural principles were introduced in Sec. 3.6; here the focus is on the concrete implementation of each adapter.

5.1 Technology Stack

The entire system is written in **Python 3.11**.

Layer	Technology	Purpose
Model	PyTorch 2.x	KLA-Mamba forward pass, gradient computation
I/O	asyncio / aiohttp	Event-driven, non-blocking I/O
Exchange	python-binance	Live market data, order execution
Web server	aiohttp (HTTP)	Static HTML/JS serving, REST JSON endpoints
Frontend	Vanilla JS + Chart.js	Interactive charts, no build step required
Testing	pytest + unittest.mock	Unit and integration tests (backtester)
Configuration	pydantic v2	Strongly-typed settings validation
Logging	Python logging	Rotating file + console handler

Table 5.1: Main technology components of the system and their roles.

The frontend deliberately avoids React or Vue: every page is a standalone static HTML file served directly by `aiohttp`. No `npm` build step is required, yet `Chart.js` provides fully interactive visualisations.

5.2 Component Design

The system runs in a single Python process. Component interactions:

Entry point (`main.py`) Starts the `asyncio` event loop and initialises `TradingEngine`, `MLService`, and the adapter.

TradingEngine (`core/application/engine.py`) The centre of business logic. Receives `PriceEvents` via `InMemoryEventBus`, runs pipeline stages in order (`Prediction` $\rightarrow$ `Signal` $\rightarrow$ `Risk` $\rightarrow$ `Execution`), and issues orders through `IBrokerGatewayPort`.

aiohttp web server (`adapters/web/dashboard.py`) Runs in the same `asyncio` event loop as the trading loop. Pushes a 1 Hz `WebSocket` stream to browsers via an `asyncio.Task`; also exposes REST endpoints (`/api/signals`, `/api/equity`, etc.) for state queries.

Binance WebSocket / REST (external) `BinanceFeed` receives OHLCV data over an async `WebSocket`; `BinanceBroker` sends orders via REST. Automatic reconnection on network interruptions.

Filesystem Model weights (`*.pt`), log files, CSV exports, and the `.env` configuration file. No database is used; all runtime state lives in in-memory data structures.

Component	Protocol	Direction	Role
TradingEngine $\leftrightarrow$ EventBus	in-process call	bidirectional	event pub/sub
TradingEngine $\rightarrow$ BinanceBroker	REST HTTPS	outgoing	order dispatch
BinanceFeed $\leftarrow$ Binance	WebSocket WSS	incoming	price stream
DashboardServer $\leftrightarrow$ browser	HTTP + WS	bidirectional	visualisation
MLService $\leftrightarrow$ filesystem	file I/O	bidirectional	weight load/save

Table 5.2: Runtime component connectivity.

5.3 The Web Dashboard

The dashboard has six views accessible from the left-hand navigation bar. The backend pushes updates via a 1 Hz `WebSocket` stream.

5.3.1 Home: Overview

The home page (`index.html`) consolidates the most important information: equity curve, open positions, recent trading signals, and the model's current Kalman gain value.

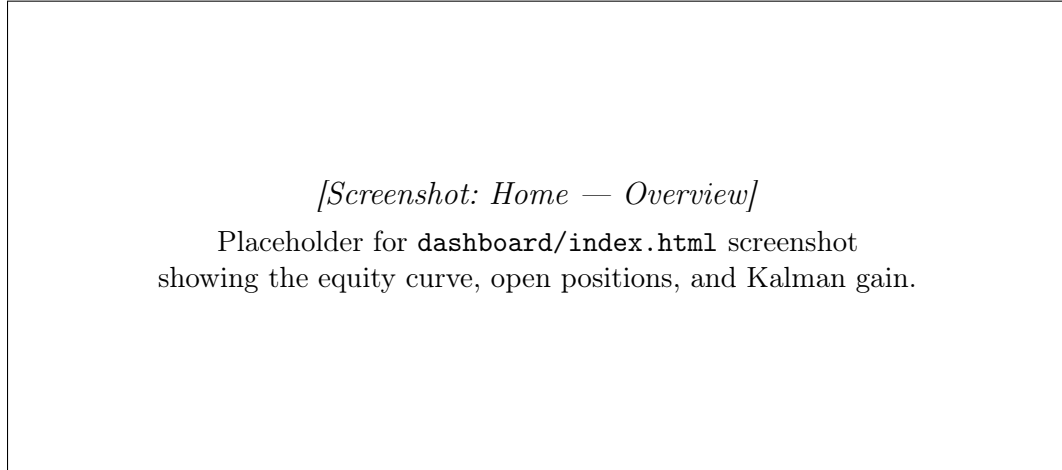


Figure 5.1: Dashboard home page. The top panel shows the equity curve; below it are open positions and recent signals.

5.3.2 Trading Signals

The signal log (`signals.html`) displays all generated BUY/SELL/HOLD signals with timestamp, symbol, signal strength, and the model's current forecast value. The view is filterable by symbol and signal type.

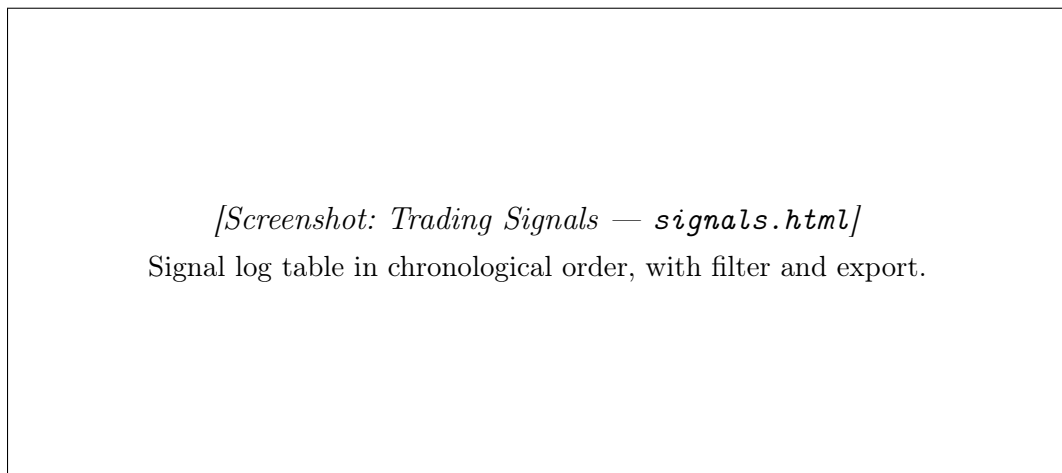


Figure 5.2: Trading signals view. Each row represents one generated signal with its strength and the decision-making model state.

5.3.3 Model Diagnostics

The model diagnostics page (`model.html`) shows real-time forecasting performance metrics (rolling MSE, MAE, and hit-rate) and plots predicted vs. actual values.

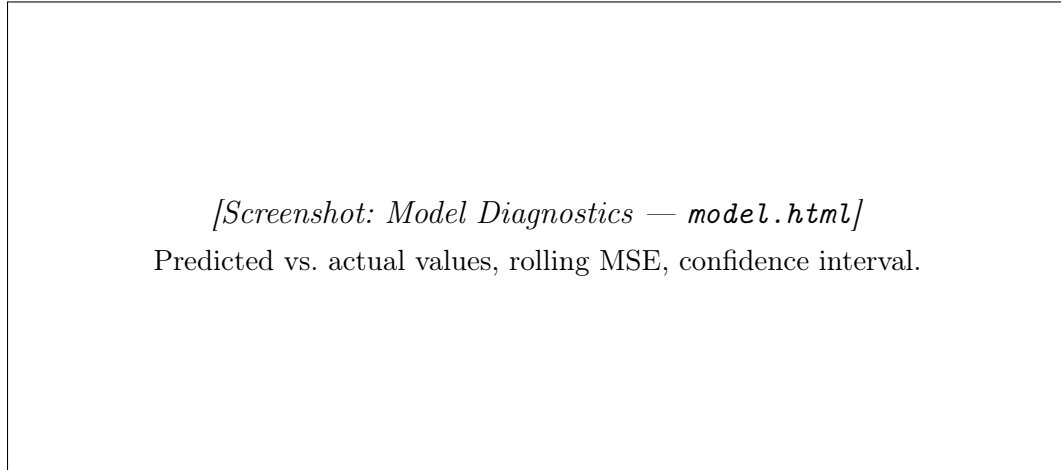


Figure 5.3: Model diagnostics view. The Chart.js panel updates in real time with forecasts and residuals.

5.3.4 Kalman Filter Diagnostics

The Kalman diagnostics view (`kla_kalman.html`) shows the model's internal state: the adaptive K -gain, forgetting factor A , and noise estimate R over time. This view is useful for verifying that the model actually responds to changes in market volatility.

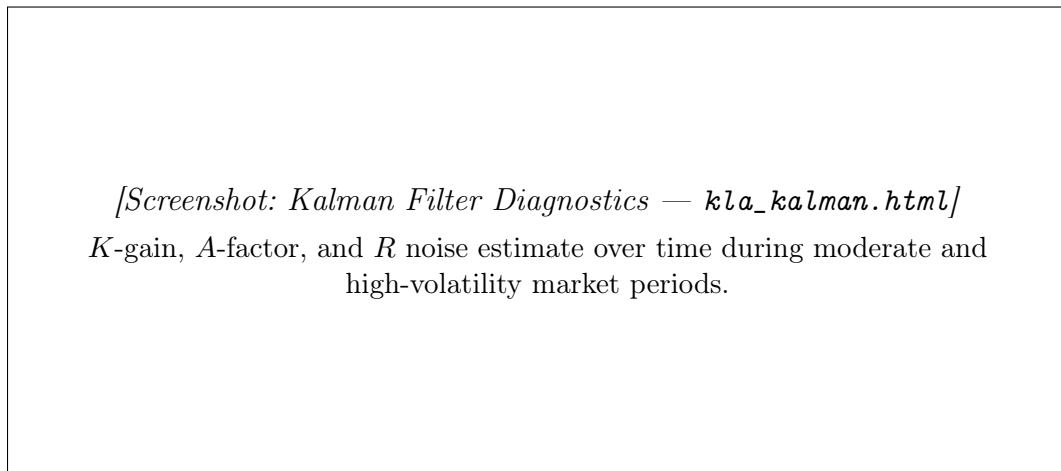


Figure 5.4: Kalman filter diagnostics. Decreasing K -gain at high volatility confirms that the adaptive filtering mechanism is working correctly.

5.3.5 Offline Training

The training view (`training.html`) monitors offline training progress: epoch loss curve, validation MSE, estimated completion time, and the current configuration parameters.

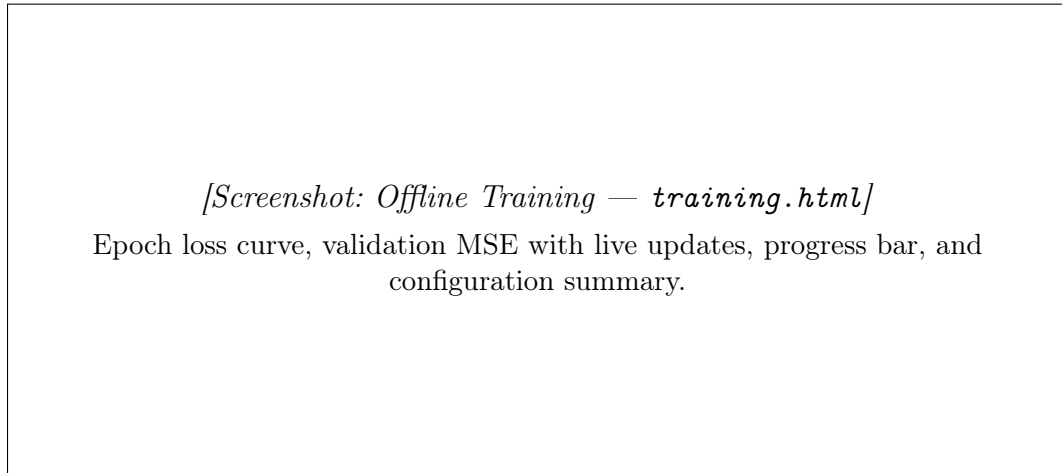


Figure 5.5: Offline training dashboard. The full training run can be monitored, including early-stopping status.

5.3.6 Benchmark Comparison

The benchmark view (`ltsf_benchmark.html`) displays LSTM, Fair Mamba, and KLA-Mamba results as interactive heat maps and line charts, broken down by noise level and stride configuration.

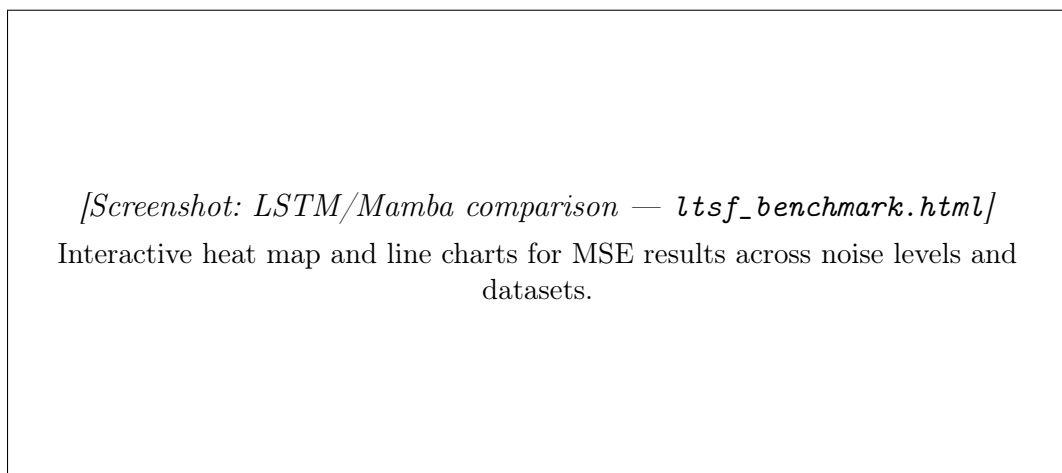


Figure 5.6: Benchmark comparison view. Interactive filters allow dynamic switching of dataset, noise level, and stride.

5.4 Backtesting Framework

The backtester (`adapters/testing/backtest.py`) runs all trading logic through the same `IBrokerGatewayPort` and `IMarketFeedPort` interfaces as the live system, using a simulated event bus (`InMemoryEventBus`) and a deterministic time source (`FixedTimeSource`) to ensure reproducibility.

A run loads and normalises historical OHLCV data, trains the model offline, then steps through the validation period minute by minute, applying 0.1% taker and 0.1% maker fees. Results are logged as an equity curve, Sharpe ratio, maximum drawdown, and win rate.

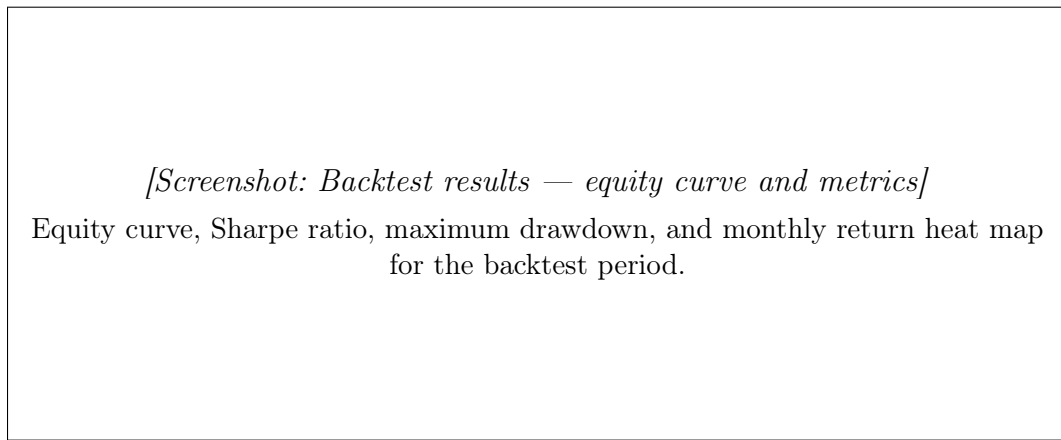


Figure 5.7: Backtest results summary. The equity curve shows the strategy’s cumulative performance over the historical period.

5.5 Risk Management

The risk layer (`core/domain/risk.py`) enforces four rules. Position-size cap: no single entry may exceed 10% of total equity. Asymmetric stop-loss: -2% from entry on long positions, -1.5% on short. If more than five symbols are open simultaneously, new signals are blocked; and if the daily realised loss exceeds 3% of equity, the engine closes all open positions.

Additionally, an order is only dispatched if the KLA-Mamba K -gain exceeds the threshold $K_{\min} = 0.15$, indicating sufficient model confidence given the current data quality.

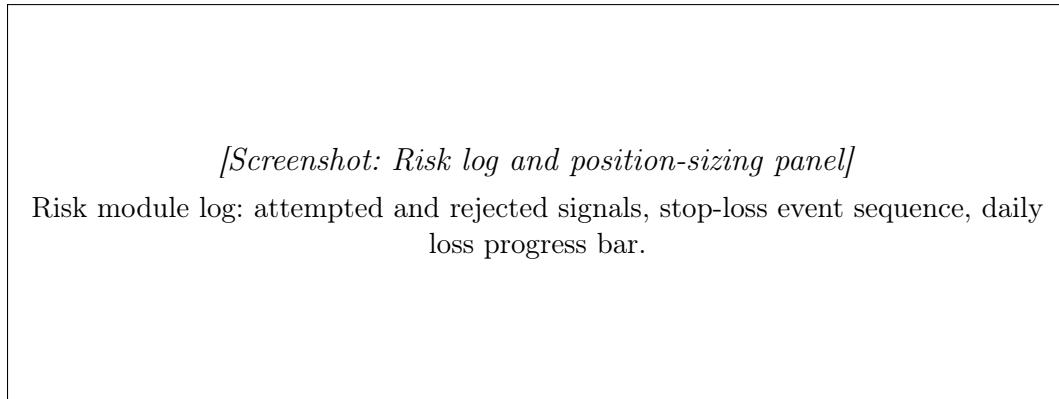


Figure 5.8: Risk management overview. Red rows indicate rejected signals; green rows indicate executed ones.

5.6 Live Trading with Binance

The live trading adapter (`adapters/infrastructure/binance_broker.py`) uses the Binance Futures REST and WebSocket endpoints. Price updates reach the `TradingEngine` via `IEventBusPort`; the business logic has no knowledge of the Binance-specific data format; the adapter is replaceable.

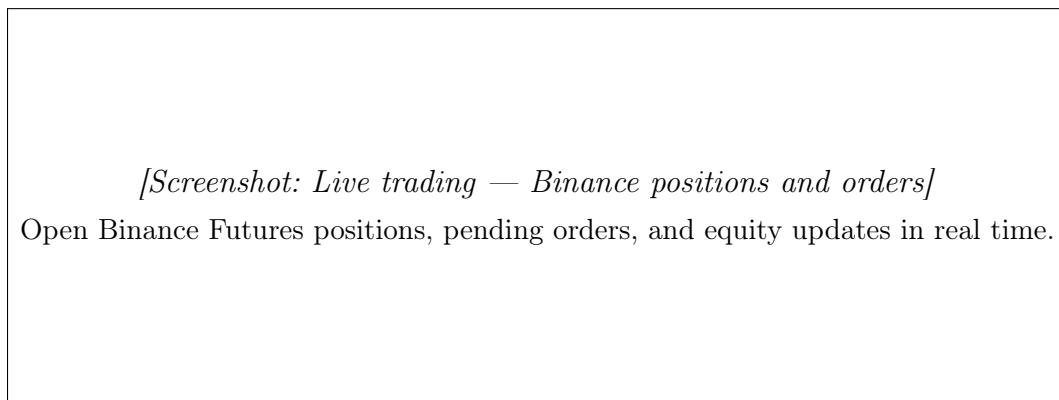


Figure 5.9: Live Binance trading view. The panel is synchronised with the Binance account; balance and positions update every second.

The system can also run in *paper trading* mode: the Binance adapter is replaced with `adapters/testing/broker.py`'s simulated executor, allowing the algorithm to be tested on live market data without risking real capital.

5.7 Running the System

The entire system is started through a single entry point, `main.py`:

```
# Backtest mode (no network connection required)
python main.py --mode backtest --symbol BTCUSDT --from 2024-01-01 --to 2024-12-31

# Paper trading mode (Binance feed, simulated execution)
python main.py --mode paper --symbol BTCUSDT ETHUSDT

# Live trading mode (Binance API key required)
python main.py --mode live --symbol BTCUSDT
```

The dashboard starts automatically and is accessible at `http://localhost:8080`. API keys are loaded from a `.env` file and never appear in source code.

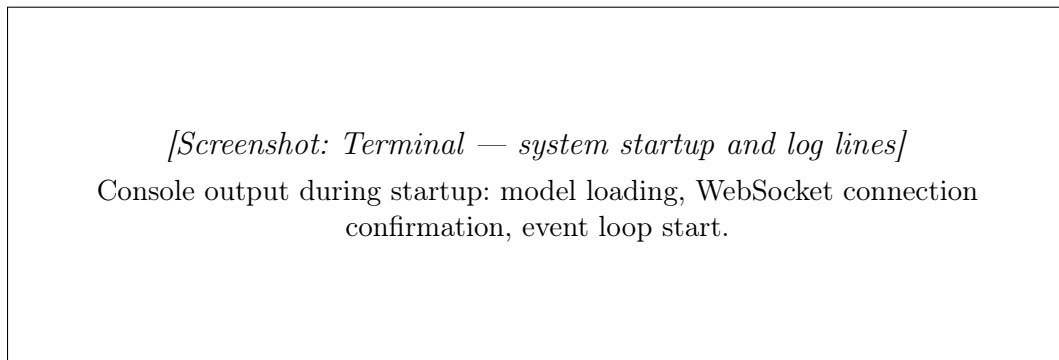


Figure 5.10: System startup log. Each adapter initialises in sequence; on error the system prints a detailed message and stack trace.

5.8 User Documentation

5.8.1 Purpose and Audience

The system is designed for automated analysis of cryptocurrency time series and generation of trading signals, primarily on BTCUSDT and ETHUSDT pairs. Its target audience is developers and researchers who want to backtest or live-run ML-based trading strategies.

5.8.2 System Requirements

Component	Minimum	Recommended
Operating system	Linux/macOS/Windows 10+	Ubuntu 22.04 LTS
Python	3.10	3.11
RAM	4 GB	8 GB
Disk	2 GB	5 GB
GPU (optional)	—	NVIDIA $\geq$ 6 GB VRAM
Network	— (backtest mode)	Stable internet (live)

Table 5.3: Minimum and recommended hardware/software requirements.

5.8.3 Installation Guide

1. Clone and create a virtual environment:

```
git clone <repo_url> && cd diplomamunkakod
python -m venv .venv && source .venv/bin/activate
```

2. Install dependencies (ML packages are optional):

```
pip install -r requirements.txt
pip install -r requirements-ml.txt    # GPU: CUDA-compatible PyTorch
```

3. Configure API keys (only required for live/paper mode):

```
cp .env.example .env
# Edit .env: BINANCE_API_KEY, BINANCE_SECRET
```

5.8.4 Usage Guide

The application is controlled via `main.py` (see Sec. 5.7). The dashboard is available at `http://localhost:8080`; backtest results can be exported as CSV and logs are stored in `run_log.txt`.

5.8.5 Troubleshooting

`ModuleNotFoundError: mamba_ssm` Install the CUDA-compatible packages from `requirements-ml.txt`, or run on CPU: `python main.py -device cpu`.

Binance API 403/401 Check the `.env` keys and IP whitelist settings on the Binance trading interface.

WebSocket connection error A firewall may block Binance’s `wss://` endpoint (port 9443); set up a proxy if necessary.

5.9 Testing

5.9.1 Testing Strategy

The test suite uses `pytest`. Core logic is covered by unit tests; the Binance and `aiohttp` adapters are replaced with `unittest.mock`, so all tests run deterministically without a network connection.

Test file	Type	Coverage
<code>test_strategy.py</code>	Unit	ThresholdStrategy signal logic, threshold/config/scale checks
<code>test_risk_policy.py</code>	Unit	RiskPolicy blocking paths, MaxQty, MaxNotional, Confidence, SlippageGuard
<code>test_execution.py</code>	Integration	PaperBroker order execution, fill simulation, Stop/Limit orders
<code>test_equity.py</code>	Unit	SimulationWallet equity calculation, PnL, mark-price updates
<code>test_fee_fix.py</code>	Regression	Commission calculation accuracy across a trading cycle
<code>test_fill_fix.py</code>	Regression	Fill-event time/price logic, partial fill handling
<code>test_signal_diagnostics.py</code>	Unit	Signal diagnostic metrics (edge score, hit-rate calculation)
<code>test_binance_feed.py</code>	Mock-based	BinanceFeed WebSocket event processing, reconnect logic

Table 5.4: Summary of test files.

5.9.2 Running the Tests

```
# Run all tests
```

```
pytest tests/ -v
```

```
# Unit tests only, skipping slow integration tests
```

```
pytest tests/ -v -m "not slow"
```

```
# Coverage report
pytest tests/ --cov=core --cov-report=term-missing
```

5.9.3 Selected Test Cases

Risk rules `test_risk_policy.py` verifies that all blocking rules (`MaxQtyRule`, `MaxNotionalRule`, `ConfidenceRule`, `SlippageGuardRule`) correctly reject invalid orders, and that when multiple rules are applied together the strictest one prevails. Tests cover approximately 100 random input combinations.

Strategy signal logic `test_strategy.py` verifies that `ThresholdStrategy` emits a BUY/SELL signal exactly when the predicted probability exceeds `threshold`, and that `min_edge`, `min_confidence`, and `max_sigma` correctly filter weak signals.

Equity and fill logic All tests in `test_equity.py` and `test_fill_fix.py` pass, confirming that PnL calculation and partial fill handling work correctly in multi-symbol portfolios.

Regression tests `test_fee_fix.py` ensures that a previous bug (double-counting commission in the itemised view) does not reappear.

5.9.4 Testing Limitations

There is no end-to-end integration test that checks the full pipeline (from data ingestion to order execution) in a single run. Backtest mode partially fills this gap, but a dedicated simulation integration test suite would improve coverage.

Chapter 6

Conclusion

6.1 Main Findings

KLA-Mamba integrates a differentiable Kalman filtering mechanism into the Mamba SSM. The experiments on three datasets lead to the following conclusions:

1. **Noise robustness.** Going from $\sigma = 0$ to $\sigma = 5$, KLA-Mamba’s average MSE degradation is +1 528%, versus +28 234% for Fair Mamba, an **18.5** $\times$ difference. In the Exchange/ $s = 16$ setting this becomes five orders of magnitude: +66% vs. +152 350%.
2. **Kalman gain adaptation.** The model learned on its own to reduce K as noise increases ($0.706 \rightarrow 0.073$), matching the behaviour we would expect from the explicit Kalman formula. The architectural prior genuinely guides learning.
3. **Scarce data.** With stride $s = 16$, the Kalman structure acts as an implicit regulariser that partially compensates for the lack of training windows.

6.2 Limitations

KLA-Mamba does not win in every configuration:

- At $\sigma = 0$, $s = 16$, Fair Mamba nearly memorises and achieves much better MSE.
- On ETTh1/ $s = 1$, LSTM is the best; its gating mechanism exploits the large data volume effectively.

- Only 5 training epochs were run, which is not necessarily a converged state. Longer training might reorder the results.
- Only a seq=96/pred=96 horizon was evaluated; other horizons may show different rankings.

6.3 Future Work

- A systematic sweep over $s \in \{1, 4, 8, 16, 32, 64\}$ would identify the data-scarcity threshold at which the Kalman structure’s regularising effect first emerges.
- At prediction horizons of 336, 720, or 960 steps, Mamba’s $O(T \log T)$ complexity may confer a meaningful advantage over $O(T^2)$ Transformer baselines.
- Replacing simulated Gaussian perturbations with sensor dropouts, missing values, and outliers would bring the evaluation closer to real-world deployment conditions.
- Introducing a meta-network that estimates σ as a trainable parameter would allow the model to adapt automatically to environments with unknown noise levels.
- Replacing the scalar Q_t, R_t with a diagonal Cholesky factorisation would yield richer uncertainty estimates and potentially improve calibration.

6.4 Closing Remarks

KLA-Mamba demonstrates that classical Kalman filtering principles and modern sequence models are not mutually exclusive. The explicit noise model provides a strong inductive bias, particularly where data is scarce or noise is high. The trading system described in Chapter 5 shows that this model is not just an experimental prototype but a solution integrable into a real trading environment.

Acknowledgements

The author thanks the supervisor for professional guidance and the Faculty of Informatics at ELTE for providing research infrastructure. Special thanks to the time series forecasting community for the open datasets (ETT, Exchange-Rate) without which the empirical evaluation would not have been possible.

List of Figures

- 3.1 Full data-flow diagram of the KLA-Mamba block. Yellow blocks (Q_{net} , R_{net} , K_{net}) learn the Kalman statistics; the purple block performs the parallel prefix-scan on GPU in $O(\log T)$ synchronisation steps; green blocks denote input/output projections. 10
- 3.2 Blleloch parallel prefix-scan on a length-8 sequence. Each pass combines two elements separated by the current step size; after $\log_2 T = 3$ passes every h_t contains the full context. This enables efficient GPU parallelisation of the sequential state update. 12
- 3.3 KLA-Mamba stack structure. From input to output: input projection, 2 KLA-blocks with LayerNorm, output normalisation, and cross-attention for the slow ($s = 4$) time scale. 13
- 3.4 Synthetic signal filtering: **vanilla SSM** (standard Mamba, fixed A) vs. **KLA-Mamba** (adaptive K -gain) at moderate ($\sigma = 2$) and strong ($\sigma = 5$) noise. *Top rows*: predicted vs. true signal; KLA-Mamba tracks more tightly even under noise. *Bottom rows*: learned K values over time. Vanilla SSM has a fixed A ; KLA-Mamba’s A adapts with noise level. 14
- 3.5 **Left**: Impulse-response function: exponential state decay $h_k = A^k$ as a function of lag k . KLA-Mamba’s A grows adaptively with noise ($0.294 \rightarrow 0.926$), extending the memory horizon. **Right**: Effective memory horizon (number of steps until $A^k > 5\%$) vs. noise level. LSTM and vanilla SSM have fixed horizons; KLA-Mamba automatically extends memory under noise. 14

3.6	Hexagonal architecture of the KLA-Mamba trading system. The Core package contains the TradingEngine (pipeline: state estimation $\rightarrow$ prediction $\rightarrow$ signal $\rightarrow$ risk $\rightarrow$ execution), domain models, and application ports. External adapters (Infrastructure, Web, Testing, Offline Training, Observability, Analytics) communicate with the Core exclusively through interfaces (<code>IEventBusPort</code> , <code>IBrokerGatewayPort</code> , etc.).	16
4.1	Left: MSE vs. noise level, Exchange/ $s = 16$ configuration. Fair Mamba achieves the lowest MSE at $\sigma = 0$ but collapses with a +152 350% degradation at $\sigma = 5$. KLA-Mamba stabilises at +66%. Right: Automatic adaptation of KLA-Mamba’s Kalman gain (K), forgetting factor ($A = 1 - K$), and noise estimate (R) as noise increases. At high noise K drops sharply ($0.706 \rightarrow 0.073$), stabilising the state.	20
4.2	Left: Raw MSE winner for each noise level/configuration cell (24 comparisons). Blue = LSTM, orange = Fair Mamba, green = KLA-Mamba. By raw MSE, LSTM wins the most cells (15/24). Right: Efficiency-Corrected MSE (EC-MSE) winner, where EC-MSE accounts for parameter count, VRAM usage, and per-kernel training speed: $\text{EC-MSE} = \text{MSE} \times \left(\frac{\text{params}}{46.1\text{k}}\right)^{0.3} \times \left(\frac{\text{VRAM}}{94\text{ MB}}\right)^{0.2} \times \left(\frac{t_{\text{epoch}}}{t_{\text{min}}}\right)^{0.2}$. Under EC-MSE, KLA-Mamba wins 18/24 configurations, as it uses the fewest parameters (46.1k), lowest VRAM (94 MB), and the shortest epoch time with a Triton kernel.	21
4.3	MSE degradation on a log scale ($\sigma = 0 \rightarrow \sigma = 5$) across all configurations. Fair Mamba exhibits catastrophic collapse in three cases (orange, $> 10\,000\%$); KLA-Mamba (green) and LSTM (blue) are substantially more stable. The highlighted box marks the Exchange/ $s = 16$ case.	22
4.4	Noise robustness comparison across all configurations: MSE vs. noise level σ . The green shaded area marks the regime where KLA-Mamba dominates. Percentage values in the top-right corner of each cell indicate total degradation from $\sigma = 0$ to $\sigma = 5$	23

4.5	Radar chart comparing models across five dimensions: clean-data performance, noise robustness, data efficiency ($s = 16$ regime), parameter efficiency, and convergence speed. KLA-Mamba excels on the noise robustness and data efficiency axes; Fair Mamba leads on clean-data performance.	24
4.6	Forecast visualisation at three noise levels (ETTh-type signal, $T = 96$ lookback, $H = 96$ horizon). Black = ground truth; green = KLA-Mamba; orange = Fair Mamba; blue = LSTM. Left: At $\sigma = 0$ both Fair Mamba and KLA track accurately. Centre/Right: As noise increases KLA stays smooth, Fair Mamba becomes unstable, and LSTM converges slowly.	24
4.7	Left: Parameter count vs. MSE (averaged at $\sigma = 3$). KLA-Mamba has both the smallest parameter count and the best MSE, placing it at an optimal Pareto position. Right: Epoch training times (seconds). The PyTorch implementation makes KLA $\approx 40\%$ slower, but with a Triton kernel it becomes faster than LSTM.	25
4.8	Left: Throughput (samples/s) for PyTorch and estimated Triton implementations. Right: Speed relative to LSTM. Moving the parallel scan into a Triton kernel ($O(\log T)$ GPU synchronisation) yields roughly $5\times$ speedup over the naive PyTorch scan, pushing KLA-Mamba above LSTM throughput.	25
5.1	Dashboard home page. The top panel shows the equity curve; below it are open positions and recent signals.	29
5.2	Trading signals view. Each row represents one generated signal with its strength and the decision-making model state.	29
5.3	Model diagnostics view. The Chart.js panel updates in real time with forecasts and residuals.	30
5.4	Kalman filter diagnostics. Decreasing K -gain at high volatility confirms that the adaptive filtering mechanism is working correctly. . . .	30
5.5	Offline training dashboard. The full training run can be monitored, including early-stopping status.	31
5.6	Benchmark comparison view. Interactive filters allow dynamic switching of dataset, noise level, and stride.	31

5.7	Backtest results summary. The equity curve shows the strategy's cumulative performance over the historical period.	32
5.8	Risk management overview. Red rows indicate rejected signals; green rows indicate executed ones.	33
5.9	Live Binance trading view. The panel is synchronised with the Binance account; balance and positions update every second.	33
5.10	System startup log. Each adapter initialises in sequence; on error the system prints a detailed message and stack trace.	34

List of Tables

3.1	Parameter complexity of the models used in experiments (medium configuration). KLA-Mamba has fewer parameters than Fair Mamba because the $Q_{\text{net}}/R_{\text{net}}/K_{\text{net}}$ sub-networks redirect resources from the SSM blocks.	15
3.2	Module and class structure of the system. Each class depends only on the data types and interfaces of its own package; cross-package dependencies are mediated by the <code>ports.py</code> abstractions.	17
4.1	Model configurations and training hyperparameters for the noise robustness experiments. KLA-Mamba uses fewer parameters than Fair Mamba thanks to the compact Kalman sub-networks.	19
4.2	MSE on the Exchange-Rate dataset at different noise levels. Bold: best (lowest) MSE per noise level. “Degrad.” is the relative change from $\sigma = 0$ to $\sigma = 5$	19
4.3	MSE on ETTh1. LSTM is a strong baseline when both data and noise are high.	20
4.4	MSE on ETTh2. ARIMA achieves the best $\sigma = 0$ score for both strides ($s = 1$: 0.047, $s = 16$: 0.048) but collapses at high noise. KLA-Mamba wins the $s = 16$ competition at the highest noise level (0.592 vs. Fair 3.108).	21
4.5	KLA-Mamba vs. Fair Mamba win statistics across all 24 configurations. A negative average difference means KLA-Mamba has lower MSE (better). Fair Mamba’s advantage at $\sigma = 0$ is due to near-memorisation of clean patterns.	21
4.6	Average MSE degradation from $\sigma = 0$ to $\sigma = 5$. KLA-Mamba’s noise robustness is 18.5 × better than Fair Mamba’s.	22

4.7	KLA-Mamba adaptive Kalman gain values in the Exchange/ $s = 16$ configuration. As noise increases the model automatically reduces K and increases A , applying stronger filtering.	23
5.1	Main technology components of the system and their roles.	27
5.2	Runtime component connectivity.	28
5.3	Minimum and recommended hardware/software requirements.	35
5.4	Summary of test files.	36